Local Citation Recommendation with Hierarchical-Attention Text Encoder and SciBERT-based Reranking

Nianlong Gu, Yingqiang Gao, and Richard H.R. Hahnloser

Institute of Neuroinformatics, University of Zurich and ETH Zurich
{nianlong,yingqiang.gao,rich}@ini.ethz.ch

Abstract. The goal of local citation recommendation is to recommend a missing reference from the local citation context and optionally also from the global context. To balance the tradeoff between speed and accuracy of citation recommendation in the context of a large-scale paper database, a viable approach is to first prefetch a limited number of relevant documents using efficient ranking methods and then to perform a fine-grained reranking using more sophisticated models. In that vein, BM25 has been found to be a tough-to-beat approach to prefetching, which is why recent work has focused mainly on the reranking step. Even so, we explore prefetching with nearest neighbor search among text embeddings constructed by a hierarchical attention network. When coupled with a SciBERT reranker fine-tuned on local citation recommendation tasks, our hierarchical Attention encoder (HAtten) achieves high prefetch recall for a given number of candidates to be reranked. Consequently, our reranker requires fewer prefetch candidates to rerank, yet still achieves state-of-the-art performance on various local citation recommendation datasets such as ACL-200, FullTextPeerRead, RefSeer, and arXiv.

Keywords: Local citation recommendation · Hierarchical attention · Document reranking.

1 Introduction

Literature discovery, such as finding relevant scientific articles, remains challenging in today's age of information overflow, largely arising from the exponential growth in both the publication record [15] and the underlying vocabulary [13]. Assistance to literature discovery can be provided with automatic citation recommendation, whereby a query text without citation serves as the input to a recommendation system and a paper worth citing as its output [9].

Citation recommendation can be dealt with either as a global retrieval problem [33,26,2] or as a local one [12,14,16]. In global citation recommendation, the query text is composed of the title and the abstract of a source paper [2]. In contrast, in local citation recommendation, the query consist of two sources of contexts [12,24]: 1) the text surrounding the citation placeholder with the information of the cited paper removed (the **local context**); and 2) the title and

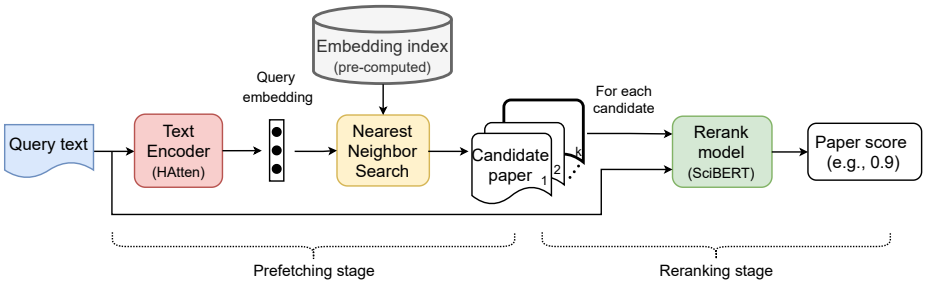


Fig. 1: Overview of our two-stage local citation recommendation pipeline.

abstract of the citing paper as the **global context**. The aim of local citation recommendation is to find the missing paper cited at the placeholder of the local context. In this paper we focus on local citation recommendation.

It is important for a local citation recommendation system to maintain a balance between accuracy (e.g., recall of the target paper among the top K recommended papers) and speed in order to operate efficiently on a large database containing millions of scientific papers. The speed-accuracy tradeoff can be flexibly dealt with using a two-step prefetching-reranking strategy: 1) A fast prefetching model first retrieves a set of candidate papers from the database; 2) a more sophisticated model then performs a fine-grained analysis of scoring candidate papers and reordering them to result in a ranked list of recommendations. In many recent studies [24,4,6,21], either (TF-IDF) [29] or BM25 [30] were used as the prefetching algorithm, which were neither fine-tuned nor taken into consideration when evaluating the recommendation performance.

In this paper, we propose a novel two-stage local citation recommendation system (Figure 1). In the prefetching stage, we make use of an embedding-based paper retrieval system, in which a siamese text encoder first pre-computes a vector-based embedding for each paper in the database. The query text is then mapped into the same embedding space to retrieve the K nearest neighbors of the query vector. To encode queries and papers of various lengths in a memory-efficient way, we design a two-layer Hierarchical Attention-based text encoder (HAtten) that first computes paragraph embeddings and then computes from the paragraph embeddings the query and document embeddings using a self-attention mechanism [34]. In the reranking step, we fine-tune the SciBERT [1] to rerank the candidates retrieved by the HAtten prefetching model.

In addition, to cope with the scarceness of large-scale training datasets in many domains, we construct a novel dataset that we distilled from 1.7 million arXiv papers. The dataset consist of 3.2 million local citation sentences along with the title and the abstract of both the citing and the cited papers. Extensive experiments on the arXiv dataset as well as on previous datasets including ACL-200 [24], RefSeer [24,6], and FullTextPeerRead [16] show that our local citation recommendation system performs better on both prefetching and reranking than the baseline and requires fewer prefetched candidates in the reranking step

thanks to higher recall of our prefetching system, which indicates that our system strikes a better speed-accuracy balance.

In total, our main contributions are summarized as follows: 1) We propose a competitive retrieval system consisting of a hierarchical-attention text encoder and a fine-tuned SciBERT reranker. 2) In evaluations of the whole pipeline, we demonstrate a well-balanced tradeoff between speed and accuracy. 3) We release our code and a large-scale scientific paper dataset[1] for training and evaluation of production-level local citation recommendation systems.

2 Related Work

Local citation recommendation was previously addressed in He et al. [12] in which a non-parametric probabilistic model was proposed to model the relevance between the query and each candidate citation. In recent years, embedding-based approaches [19,10] have been proposed to more flexibly capture the resemblance between the query and the target according to the cosine distance or the Euclidean distance between their embeddings. Jeong et al. [16] proposed a BERT-GCN model in which they used Graph Convolutional Networks [18] (GCN) and BERT [5] to compute for each paper embeddings of the citation graph and the query context, which they fed into a feed-forward network to estimate relevance. The BERT-GCN model was evaluated on small datasets of only thousands of papers, partly due to the high cost of computing the GCN, which limited its scalability for recommending citations from large paper databases. Although recent studies [24,4,6,21] adopted the prefetching-reranking strategy to improve the scalability, the prefetch part (BM25 or TF-IDF) only served for creating datasets for training and evaluating the reranking model, since the target cited paper was added manually if it was not retrieved by the prefetch model, i.e. the recall of the target among the candidate papers was set to 1. Therefore, these recommendation systems were evaluated in an artificial situation with an ideal prefetching model that in reality does not exist.

Supervised methods for citation recommendation rely on the availability of numerous labeled data for training. It is challenging to assemble a dataset for local citation recommendation due to the need of parsing the full text of papers to extract the local contexts and finding citations that are also available in the dataset, which eliminates a large bulk of data. Therefore, existing datasets on local citation recommendation are usually limited in size. For example, the ACL-200 [24] and the FullTextPeerRead [16] contain only thousands of papers. One of the largest datasets is RefSeer used in Medić and Šnajder [24], which contains 0.6 million papers in total, but this dataset is not up-to-date as it only contains papers prior to 2015. Although unarXive [31], a large dataset for citation recommendation, exists, this dataset does not meet the needs of our task because: 1) papers in unarXive are not parsed in a structured manner. For example, the abstract is not separated from the full text, which makes it

[1] Our code and data are available at https://github.com/nianlonggu/Local-Citation-Recommendation.

difficult to construct a global context in our experiments; 2) the citation context is usually a single sentence containing a citation marker, even if the sentence does not contain sufficient contextual information, e.g., "For details, see [#]". These caveats motivate the creation of a novel dataset of high quality.

3 Proposed Dataset

We create a new dataset for local citation recommendation using arXiv papers contained in S2ORC [22], a large-scale scientific paper corpus. Each paper in S2ORC has an identifier of the paper source, such as arXiv or PubMed. Using this identifier, we first obtain all arXiv papers with available titles and abstracts. The title and abstract of each paper are required because they are used as the global context of a query from that paper or as a representation of the paper's content when the latter is a candidate to be ranked. From the papers we then extract the local contexts by parsing those papers for which the full text is available: For each reference in the full text, if the cited paper is also available in the arXiv paper database, we replace the reference marker such as "[#]" or "XXX et al." with a special token such as "CIT", and collect 200 characters surrounding the replaced citation marker as the local context. Note that we "cut off" a word if it lied on the 200-character boundary, following the setting of the ACL-200 and the RefSeer datasets proposed in Medić and Šnajder [24].

Dataset	Number of local contexts			Number of papers	publication years
	Train	Val	Test		
ACL-200	30,390	9,381	9,585	19,776	2009 – 2015
FullTextPeerRead	9,363	492	6,814	4,837	2007 – 2017
RefSeer	3,521,582	124,911	126,593	624,957	– 2014
arXiv (Ours)	2,988,030	112,779	104,401	1,661,201	1991 – 2020

Table 1: Statistics of the datasets for local citation recommendation.

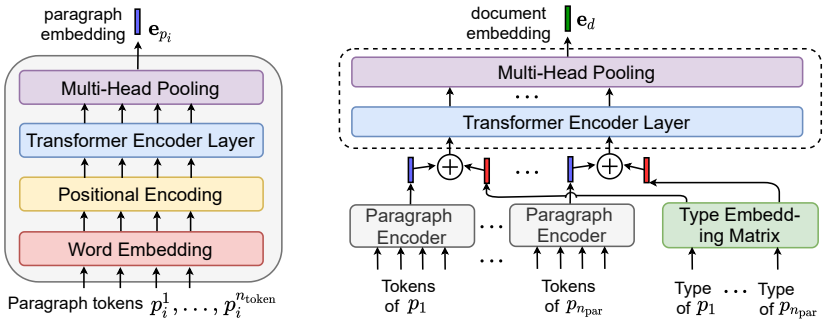
Table 1 shows the statistics of the created arXiv dataset and the comparison with existing datasets used in this paper. As the most recent contexts available in the arXiv dataset is from April 2020, we use the contexts from 1991 to 2019 as the training set, the contexts from January 2020 to February 2020 as the validating set, and the contexts from March 2020 to April 2020 as the test set. The sizes of the arXiv training, validating, and testing sets are comparable to RefSeer, one of the largest existing datasets, whereas our arXiv dataset contains a much larger number of papers, and there are more recently published papers available in the arXiv dataset. These features make the arXiv dataset a more challenging and up-to-date test bench.

4 Approach

Our two-stage telescope citation recommendation system is similar to that of Bhagavatula et al. [2], composed of a fast **prefetching** model and a slower **reranking** model.

4.1 Prefetching Model

The prefetching model scores and ranks all papers in the database to fetch a rough initial subset of candidates. We designed a representation-focused ranking model [11] that computes a query embedding for each input query and ranks each candidate document according to the cosine similarity between the query embedding and the pre-computed document embedding.



(a) Paragraph Encoder. (b) Document Encoder.

Fig. 2: The Hierarchical-Attention text encoder (HAtten) used in the prefetching step is composed of a paragraph encoder (a) and a document encoder (b).

The core of the prefetching model is a light-weight text encoder that efficiently computes the embeddings of queries and candidate documents. As shown in Figure 2, the encoder processes each document or query in a two-level hierarchy, consisting of two components: a paragraph encoder and a document encoder.

Paragraph Encoder For each paragraph p_i in the document, the paragraph encoder (Figure 2a) takes as input the token sequence $p_i = [w_1, \ldots, w_{n_i}]$ composed of n_i tokens (words) to output the paragraph embedding $\boldsymbol{e}_{p_i}$ as a single vector. In order to incorporate positional information of the tokens, the paragraph encoder makes use of positional encoding. Contextual information is encoded with a single transformer encoder layer following the configuration in Vaswani et al. [34], Figure 2a. To obtain a single fixed-size embedding $\boldsymbol{e}_p$ from a variably sized paragraph, the paragraph encoder processes the output of the transformer encoder layer with a multi-head pooling layer [20] with trainable weights. Let

$x_k \in \mathbb{R}^d$ be the output of the transformer encoder layer for token w_k in a paragraph p_i. For each head $j \in \{1, \ldots, n_{\text{head}}\}$ in the multi-head pooling layer, we first compute a value vector $v_k^j \in \mathbb{R}^{d/n_{\text{head}}}$ as well as an attention score $\hat{a}_k^j \in \mathbb{R}$ associated with that value vector:

$$v_k^j = \text{Linear}_v^j(x_k), \quad a_k^j = \text{Linear}_a^j(x_k), \quad \hat{a}_k^j = \frac{\exp a_k^j}{\sum_{m=1}^{n_{\text{token}}} \exp a_m^j}, \tag{1}$$

where Linear() denotes a trainable linear transformation. The weighted value vector $\hat{v}^j$ then results from the sum across all value vectors weighed by their corresponding attention scores: $\hat{v}^j = \sum_{m=1}^{n_{\text{par}}} \hat{a}_m^j v_m^j$. The final paragraph embedding $\boldsymbol{e}_p$ is constructed from the weighted value vectors $\hat{v}^j$ of all heads by a ReLU activation [25] followed by a linear transformation:

$$\boldsymbol{e}_p = \text{Linear}_p(\text{ReLU}(\text{Concat}(\hat{v}^1, \ldots, \hat{v}^{n_{\text{head}}}))). \tag{2}$$

Document Encoder In order to encode documents with two fields given by the title and the abstract, or to encode queries given by three fields: the local context, the title, and the abstract of the citing paper, we treat each field (local context, title, and abstract) as a paragraph. For a document of n_{par} paragraphs $d = [p_1, \ldots, p_{n_{\text{par}}}]$, we first compute the embeddings of all paragraphs p_i.

Not all fields and paragraphs are treated equally in our document encoder. To allow the document encoder to distinguish between fields, we introduce a *paragraph type* variable, which refers to the field type from which the paragraph originates. We distinguish between three paragraph types: the title, the abstract, and the local context. Each type is associated with a learnable type embedding that has the same dimension as the paragraph embedding. Inspired by the BERT model [5], we produce a type-aware paragraph embedding by adding the type embedding of the given paragraph to the corresponding paragraph embedding (Figure 2b). All type-aware paragraph embeddings are then fed into a transformer encoder layer followed by a multi-head pooling layer (of identical structures as the ones in the paragraph encoder), which then results in the final document embedding $\boldsymbol{e}_d$.

Prefetched document candidates The prefetched document candidates are found by identifying the K nearest document embeddings to the query embedding in terms of cosine similarity. The ranking is performed using a brute-force nearest neighbor search among all document embeddings as shown in Figure 1.

4.2 Reranking Model

The reranking model performs a fine-grained comparison between a query q (consisting of a local and a global context) and each prefetched document candidate (its title and the abstract). The relevance scores of the candidates constitute the final output of our model. We design a reranker based on SciBERT [1], which is

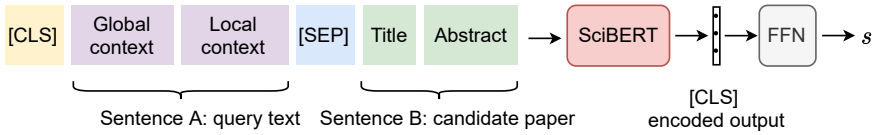


Fig. 3: Structure of our SciBERT Reranker.

a BERT model [5] trained on a large-scale corpus of scientific articles. The input of the SciBERT reranker has the following format: "[CLS] Sentence A [SEP] Sentence B", where sentence A is the concatenation of the global context (title and abstract of the citing paper) and the local context of the query, and sentence B is the concatenation of the title and the abstract of the candidate paper to be scored, Figure 3. The SciBERT-encoded vector for the "[CLS]" token is then fed into a feed-forward network that outputs the relevance score $s \in [0, 1]$ provided via a sigmoid function.

4.3 Loss Function

We use a triplet loss both to train our HAtten text encoder for prefetching and to finetune the SciBERT reranker. The triplet loss is based on the similarity $s(q, d)$ between the query q and a document d. For the prefetching step, $s(q, d)$ is given by the cosine similarity between the query embedding v_q and the document embedding v_d, both computed with the HAtten encoder. For the reranking step, $s(q, d)$ is given by the relevance score computed by the SciBERT reranker. In order to maximize the relevance score between the query q and the cited document d_+ (the positive pair (q, d_+)) and to minimize the score between q and any non-cited document d_- (a negative pair (q, d_-)), we minimize the triplet loss:

$$\mathcal{L} = \max[s(q, d_-) - s(q, d_+) + m, 0] \tag{3}$$

where the margin $m > 0$ sets the span over which the loss is sensitive to the similarity of negative pairs.

For fast convergence during training, it is important to select effective triplets for which $\mathcal{L}$ in Equation (3) is non-zero [32], which is particularly relevant for the prefetching model, since for each query there is only a single positive document but millions of negative documents (e.g., on the arXiv dataset). Therefore, we employ negative and positive mining strategies to train our HAtten encoder, described as follows.

Negative mining Given a query q, we use HAtten's current checkpoint to prefetch the top K_n candidates excluding the cited paper. The HAtten embedding of these prefetched non-cited candidates have high cosine similarity to the HAtten embedding of the query. To increase the similarity between the query and the cited paper while suppressing the similarity between the query and these non-cited candidates, we use the cited paper as the positive document and select the negative document from these K_n overly similar candidates.

Positive mining Among the prefetched non-cited candidates, the documents with objectively high textual similarity (e.g. measured by word overlapping, such as the Jaccard index [2]) to the query were considered relevant to the query, even if they were not cited. These textually relevant candidate documents should have a higher cosine similarity to the query than randomly selected documents. Therefore, in parallel with the negative mining strategy, we also select positive documents from the set of textually relevant candidates and select negative documents by random sampling from the entire dataset.

The checkpoint of the HAtten model is updated every N_{iter} training iterations, at which point the prefetched non-cited and the textually relevant candidates for negative and positive mining are updated as well.

In contrast, when fine-tuning SciBERT for reranking, the reranker only needs to rerank the top K_r prefetched candidates. This allows for a simpler triplet mining strategy, which is to select the cited paper as the positive document and randomly selecting a prefetched non-cited papers as the negative document.

5 Experiments

Implementation Details In the prefetching step, we used as word embeddings of the HAtten text encoder the pre-trained 200-dimensional GloVe embeddings [28], which were kept fixed during training. There are 64 queries in a mini-batch, each of which was accompanied by 1 cited paper, 4 non-cited papers randomly sampled from the top $K_n = 100$ prefetched candidates, and 1 randomly sampled paper from the whole database, which allow us to do negative and positive mining with the mini-batch as described in Section 4.3. The HAtten's checkpoint was updated every $N_{\text{iter}} = 5000$ training iterations.

In the reranking step, we initialized the SciBERT reranker with the pre-trained model provided in Beltagy et al. [1]. The feed-forward network in Figure 3 consisting of a single linear layer was randomly initialized. Within a mini-batch there was 1 query, 1 cited paper (positive sample), and 62 documents (negative samples) randomly sampled from the top $K_r = 2000$ prefetched non-cited documents. In the triplet loss function the margin m was set to 0.1.

We used the Adam optimizer [17] with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. In the prefetching step, the learning rate was set to $\alpha = 1e^{-4}$ and the weight decay to $1e^{-5}$, while in the reranking step these were set to $1e^{-5}$ and to $1e^{-2}$ for fine-tuning SciBERT, respectively. The models were trained on eight NVIDIA GeForce RTX 2080 Ti 11GB GPUs and tested on two Quadro RTX 8000 GPUs.

Evaluation Metrics We evaluated the recommendation performance using the Mean Reciprocal Rank (MRR) [35] and the Recall@K (R@K for short), consistent with previous work [24,8,16]. The MRR measures the reciprocal rank of the actually cited paper among the recommended candidates, averaged over multiple queries. The R@K evaluates the percentage of the cited paper appearing in the top K recommendations.

Baselines In the prefetching step, we compare our HAtten with the following baselines: BM25, Sent2vec [27], and NNSelect [2]. BM25 was used as the prefetch-

ing method in previous works [24,6,21]. Sent2vec is an unsupervised text encoder which computes a text embedding by averaging the embeddings of all words in the text. We use the 600-dim Sent2vec pretrained on Wikipedia. NNSelect [2] computes text embeddings also by averaging, and the trainable parameters are the magnitudes of word embeddings that we trained on each dataset using the same training configuration as our HAtten model.

In the reranking step, we compare our fine-tuned SciBERT reranker with the following baselines: 1) a Neural Citation Network (NCN) with an encoder-decoder architecture [6,7]; 2) DualEnh and DualCon [24] that score each candidate using both semantic information and bibliographic information and 3) BERT-GCN [16]. Furthermore, to analyze the influence on ranking performance of diverse pretraining corpuses for BERT, we compared our SciBERT reranker with a BERT reranker that was pretrained on a non-science specific corpus [5] and then fine-tuned on the reranking task.

For a fair performance comparison of our reranker with those of other works, we adopted the prefetching strategies from each of these works. On ACL-200 and RefSeer, we tested our SciBERT reranker on the test sets provided in Medić and Šnajder [24]. For each query in the test set, we prefetched n ($n = 2000$ for ACL-200 and $n = 2048$ for RefSeer) candidates using BM25, and manually added the cited paper as candidate if it was not found by BM25. In other words, we constructed our test set using an "oracle-BM25" with R@$n = 1$. On the FullTextPeerRead dataset, we used our SciBERT reranker to rank all papers in the database without prefetching, in line with the setting in BERT-GCN [16]. On our newly proposed arXiv dataset, we fetched the top 2000 candidates for each query in the test set using the "oracle-BM25" as introduced above.

6 Results and Discussion

In this section, we first present the evaluation results of our prefetching and reranking models separately and compare them with baselines. Then, we evaluate the performance of the entire prefetching-reranking pipeline, and analyze the influence of the number of prefetched candidates to be reranked on the overall recommendation performance.

6.1 Prefetching Results

Our HAtten model significantly outperformed all baselines (including the strong baseline BM25, Table 2) on the ACL-200, RefSeer and the arXiv datasets, evaluated in terms of MRR and R@K. We observed that, first, for larger K, such as $K = 200, 500, 1000, 2000$, the improvement of R@K with respect to the baselines is more pronounced on all four datasets, where the increase is usually larger than 0.1, which means that the theoretical upper bound of the final reranking recall will be higher when using our HAtten prefetching system. Second, the improvements of R@K on large datasets such as RefSeer and arXiv are more prominent than on small datasets such as ACL-200 and FullTextPeerRead, which fits well

Dataset	Model	avg. prefetch time (ms)	MRR	R@10	R@100	R@200	R@500	R@1000	R@2000
ACL-200	BM25	9.9 ± 20.1	0.138	0.263	0.520	0.604	0.712	0.791	0.859
	Sent2vec	1.8 ± 19.5	0.066	0.127	0.323	0.407	0.533	0.640	0.742
	NNSelect	1.8 ± 3.8	0.076	0.150	0.402	0.498	0.631	0.722	0.797
	HAtten	2.7 ± 3.8	**0.148***	**0.281***	**0.603***	**0.700***	**0.803***	**0.870***	**0.924***
FullText-PeerRead	BM25	5.1 ± 18.6	**0.185***	**0.328***	0.609	0.694	0.802	0.877	0.950
	Sent2vec	1.7 ± 19.6	0.121	0.215	0.462	0.561	0.694	0.794	0.898
	NNSelect	1.7 ± 4.8	0.130	0.255	0.572	0.672	0.790	0.869	0.941
	HAtten	2.6 ± 4.9	0.167	0.306	**0.649***	**0.750***	**0.870***	**0.931***	**0.976***
RefSeer	BM25	216.2 ± 84.9	0.099	0.189	0.398	0.468	0.561	0.631	0.697
	Sent2vec	6.0 ± 20.9	0.061	0.111	0.249	0.306	0.389	0.458	0.529
	NNSelect	4.3 ± 5.5	0.044	0.080	0.197	0.250	0.331	0.403	0.483
	HAtten	6.2 ± 7.3	**0.115***	**0.214***	**0.492***	**0.589***	**0.714***	**0.795***	**0.864***
arXiv	BM25	702.2 ± 104.7	0.118	0.222	0.451	0.529	0.629	0.700	0.763
	Sent2vec	11.3 ± 13.6	0.072	0.131	0.287	0.347	0.435	0.501	0.571
	NNSelect	6.9 ± 4.6	0.042	0.079	0.207	0.266	0.359	0.437	0.520
	HAtten	8.0 ± 4.5	**0.124***	**0.241***	**0.527***	**0.619***	**0.734***	**0.809***	**0.871***

Table 2: Prefetching performance. For Tables 2-4, the asterisks "*" indicate statistical significance ($p < 0.05$) in comparison with the closest baseline in a t-test. The red color indicates a large (> 0.8) Cohen's d effect size [3].

with the stronger need of a prefetching-reranking pipeline on large datasets due to the speed-accuracy tradeoff.

The advantage of our HAtten model is also reflected in the average prefetching time. As shown in Table 2, the HAtten model shows faster prefetching than BM25 on large datasets such as RefSeer and arXiv. This is because for HAtten, both text encoding and embedding-based nearest neighbor search can be accelerated by GPU computing, while BM25[2] benefits little from GPU acceleration because it is not vector-based. Although other embedding-based baselines such as Sent2vec and NNSelect also exhibit fast prefetching, our HAtten prefetcher has advantages in terms of both speed and accuracy.

6.2 Reranking Results

Model	ACL-200		FullTextPeerRead		RefSeer		arXiv	
	MRR	R@10	MRR	R@10	MRR	R@10	MRR	R@10
NCN	-	-	-	-	0.267	0.291	-	-
DualCon	0.335	0.647	-	-	0.206	0.406	-	-
DualEnh	0.366	0.703	-	-	0.280	0.534	-	-
BERT-GCN	-	-	0.418	0.529	-	-	-	-
BERT Reranker	0.482	0.736	0.458	0.706	0.309	0.535	0.226	0.399
SciBERT Reranker	**0.531***	**0.779***	**0.536***	**0.773***	**0.380***	**0.623***	**0.278***	**0.475***

Table 3: Comparison of reranking performance on four datasets.

[2] We implemented the Okapi BM25 [23], with $k = 1.2, b = 0.75$.

As shown in Table 3, the SciBERT reranker significantly outperformed previous state-of-the-art models on the ACL-200, the RefSeer, and the FullTextPeerRead datasets. We ascribe this improvement to BERT's ability of capturing the semantic relevance between the query text and the candidate text, which is inherited from the "next sentence prediction" pretraining task that aims to predict if two sentences are consecutive. The SciBERT reranker also performed significantly better than its BERT counterpart, suggesting that large language models pretrained on scientific papers' corpus are advantageous for citation reranking.

6.3 Performance of entire Recommendation Pipeline

Dataset	Recommendation Pipeline		Number of reranked candidates				
	Prefetch	Rerank	100	200	500	1000	2000
ACL-200	BM25	$\text{SciBERT}_{\text{BM25}}$	0.457	0.501	0.549	0.577	0.595
	HAtten	$\text{SciBERT}_{\text{HAtten}}$	**0.513***	**0.560***	**0.599***	**0.619***	**0.633***
FullText-PeerRead	BM25	$\text{SciBERT}_{\text{BM25}}$	0.527	0.578	0.639	0.680	0.720
	HAtten	$\text{SciBERT}_{\text{HAtten}}$	**0.586***	**0.651***	**0.713***	**0.739***	**0.757***
RefSeer	BM25	$\text{SciBERT}_{\text{BM25}}$	0.305	0.332	0.365	0.380	0.383
	HAtten	$\text{SciBERT}_{\text{HAtten}}$	**0.362***	**0.397***	**0.428***	**0.443***	**0.454***
arXiv	BM25	$\text{SciBERT}_{\text{BM25}}$	0.333	0.357	0.377	0.389	0.391
	HAtten	$\text{SciBERT}_{\text{HAtten}}$	**0.374***	**0.397***	**0.425***	**0.435***	**0.439***

Table 4: The performance of the entire prefetching-reranking pipeline, measured in terms of R@10 of the final reranked document list. We varied the number of prefetched candidates for reranking. For the RefSeer and arXiv datasets, we evaluated performance on a subset of 10K examples from the test set due to computational resource limitations.

The evaluation in Section 6.2 only reflects the reranking performance because the prefetched candidates are obtained by an oracle-BM25 that guarantees inclusion of the cited paper among the prefetched candidates, even though such an oracle prefetching model does not exist in reality. Evaluating recommendation systems in this context risks overestimating the performance of the reranking part and underestimating the importance of the prefetching step. To better understand the recommendation performance in real-world scenarios, we compared two pipelines: 1) BM25 prefetching + SciBERT reranker fine-tuned on BM25-prefetched candidates, denoted as $\text{SciBERT}_{\text{BM25}}$; 2) HAtten prefetching + $\text{SciBERT}_{\text{HAtten}}$ reranker fine-tuned on HAtten-prefetched candidates. We evaluated recommendation performance by R@10 of the final reranked document list and monitored the dependence of R@10 on the number of prefetched candidates for reranking.

As shown in Table 4, the HAtten-based pipeline achieves competitive performance, even when compared with the oracle prefetching model in Section 6.2. In particular, on the FullTextPeerRead dataset, using our HAtten-based pipeline,

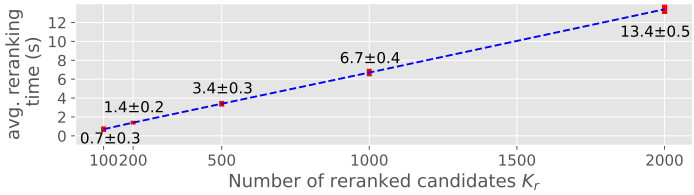


Fig. 4: The reranking time of the SciBERT reranker linearly increases with the number of reranked candidates K_r, tested on arXiv. In comparison, the prefetching time is invariant of K_r, as the prefetcher always scores and ranks all documents in the database to fetch the candidates to be reranked.

we only need to rerank 100 prefetched candidates to outperform the BERT-GCN model (Table 3) that reranked all 4.8k papers in the database.

Compared to the BM25-based pipeline, our HAtten-based pipeline achieves significantly higher R@10 for any given number of prefetched candidates. Our reranker needs to rerank only 200 to 500 candidates to match the recall score of the BM25-based pipeline needing to rerank 2000 candidates. For large datasets like RefSeer and arXiv, such improvements are even more pronounced. Our pipeline achieves a much higher throughput. For example, on the arXiv dataset, in order to achieve an overall R@10=0.39, the BM25-based pipeline takes 0.7 s (Table 2) to prefetch 2000 candidates and it takes another 13.4 s (Figure 4) to rerank them, which in total amounts to 14.1 s. In contrast, the HAtten-based pipeline only takes 8 ms to prefetch 200 candidates and 1.4 s to rerank them, which amounts to 1.4 s. This results in a 90% reduction of overall recommendation time achieved by our pipeline.

These findings provide clear evidence that a better-performing prefetching model is critical to a large-scale citation recommendation pipeline, as it allows the reranking model to rerank fewer candidates while maintaining recommendation performance, resulting in a better speed-accuracy tradeoff.

7 Conclusion

The speed-accuracy tradeoff is crucial for evaluating recommendation systems in real-world settings. While reranking models have attracted increasing attention for their ability to improve recall and MRR scores, in this paper we show that it is equally important to design an efficient and accurate prefetching system. In this regard, we propose the HAtten-SciBERT recommendation pipeline, in which our HAtten model effectively prefetches a list of candidates with significantly higher recall than the baseline, which allows our fine-tuned SciBERT-based reranker to operate on fewer candidates with better speed-accuracy tradeoff. Furthermore, by releasing our large-scale arXiv-based dataset, we provide a new testbed for research on local citation recommendation in real-world scenarios.

Acknowledgements

We acknowledge support from the Swiss National Science Foundation (grant 31003A_182638) and the NCCR Evolving Language, Swiss National Science Foundation Agreement No. 51NF40_180888. We also thank the anonymous reviewers for their useful comments.

References

1. Beltagy, I., Lo, K., Cohan, A.: SciBERT: A pretrained language model for scientific text. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). pp. 3615–3620. Association for Computational Linguistics, Hong Kong, China (Nov 2019). https://doi.org/10.18653/v1/D19-1371, https://www.aclweb.org/anthology/D19-1371
2. Bhagavatula, C., Feldman, S., Power, R., Ammar, W.: Content-based citation recommendation. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). pp. 238–251. Association for Computational Linguistics, New Orleans, Louisiana (Jun 2018). https://doi.org/10.18653/v1/N18-1022, https://www.aclweb.org/anthology/N18-1022
3. Cohen, J.: Statistical power analysis for the behavioral sciences. Academic press (2013)
4. Dai, T., Zhu, L., Wang, Y., Carley, K.M.: Attentive stacked denoising autoencoder with bi-lstm for personalized context-aware citation recommendation. IEEE/ACM Trans. Audio, Speech and Lang. Proc. **28**, 553–568 (Jan 2020). https://doi.org/10.1109/TASLP.2019.2949925, https://doi.org/10.1109/TASLP.2019.2949925
5. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). pp. 4171–4186. Association for Computational Linguistics, Minneapolis, Minnesota (Jun 2019). https://doi.org/10.18653/v1/N19-1423, https://www.aclweb.org/anthology/N19-1423
6. Ebesu, T., Fang, Y.: Neural citation network for context-aware citation recommendation. In: Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval. p. 1093–1096. SIGIR '17, Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3077136.3080730, https://doi.org/10.1145/3077136.3080730
7. Färber, M., Klein, T., Sigloch, J.: Neural citation recommendation: A reproducibility study. In: BIR@ECIR (2020)
8. Färber, M., Sampath, A.: Hybridcite: A hybrid model for context-aware citation recommendation. In: Proceedings of the ACM/IEEE Joint Conference on Digital Libraries in 2020. p. 117–126. JCDL '20, Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3383583.3398534, https://doi.org/10.1145/3383583.3398534

9. Färber, M., Jatowt, A.: Citation recommendation: approaches and datasets. International Journal on Digital Libraries **21**(4), 375–405 (Aug 2020). https://doi.org/10.1007/s00799-020-00288-2, http://dx.doi.org/10.1007/s00799-020-00288-2
10. Gökçe, O., Prada, J., Nikolov, N.I., Gu, N., Hahnloser, R.H.: Embedding-based scientific literature discovery in a text editor application. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. pp. 320–326. Association for Computational Linguistics, Online (Jul 2020). https://doi.org/10.18653/v1/2020.acl-demos.36, https://www.aclweb.org/anthology/2020.acl-demos.36
11. Guo, J., Fan, Y., Pang, L., Yang, L., Ai, Q., Zamani, H., Wu, C., Croft, W.B., Cheng, X.: A deep look into neural ranking models for information retrieval. Information Processing & Management p. 102067 (2019)
12. He, Q., Pei, J., Kifer, D., Mitra, P., Giles, L.: Context-aware citation recommendation. In: Proceedings of the 19th international conference on World wide web. pp. 421–430 (2010)
13. Herdan, G.: Type-token mathematics, vol. 4. Mouton (1960)
14. Huang, W., Kataria, S., Caragea, C., Mitra, P., Giles, C.L., Rokach, L.: Recommending citations: translating papers into references. In: Proceedings of the 21st ACM international conference on Information and knowledge management. pp. 1910–1914 (2012)
15. Hunter, L., Cohen, K.B.: Biomedical language processing: what's beyond pubmed? Molecular cell **21**(5), 589–594 (2006)
16. Jeong, C., Jang, S., Park, E.L., Choi, S.: A context-aware citation recommendation model with BERT and graph convolutional networks. Scientometrics **124**(3), 1907–1922 (2020). https://doi.org/10.1007/s11192-020-03561-y, https://doi.org/10.1007/s11192-020-03561-y
17. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: Bengio, Y., LeCun, Y. (eds.) 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings (2015), http://arxiv.org/abs/1412.6980
18. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings. OpenReview.net (2017), https://openreview.net/forum?id=SJU4ayYgl
19. Kobayashi, Y., Shimbo, M., Matsumoto, Y.: Citation recommendation using distributed representation of discourse facets in scientific articles. In: Proceedings of the 18th ACM/IEEE on Joint Conference on Digital Libraries. p. 243–251. JCDL '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3197026.3197059, https://doi.org/10.1145/3197026.3197059
20. Liu, Y., Lapata, M.: Hierarchical transformers for multi-document summarization. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. pp. 5070–5081. Association for Computational Linguistics, Florence, Italy (Jul 2019). https://doi.org/10.18653/v1/P19-1500, https://www.aclweb.org/anthology/P19-1500
21. Livne, A., Gokuladas, V., Teevan, J., Dumais, S.T., Adar, E.: Citesight: Supporting contextual citation recommendation using differential search. In: Proceedings of the 37th International ACM SIGIR Conference on Research & Development

in Information Retrieval. p. 807–816. SIGIR '14, Association for Computing Machinery, New York, NY, USA (2014). https://doi.org/10.1145/2600428.2609585, https://doi.org/10.1145/2600428.2609585
22. Lo, K., Wang, L.L., Neumann, M., Kinney, R., Weld, D.S.: S2orc: The semantic scholar open research corpus. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. pp. 4969–4983 (2020)
23. Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press, Cambridge, UK (2008), http://nlp.stanford.edu/IR-book/information-retrieval-book.html
24. Medić, Z., Snajder, J.: Improved local citation recommendation based on context enhanced with global information. In: Proceedings of the First Workshop on Scholarly Document Processing. pp. 97–103. Association for Computational Linguistics, Online (Nov 2020). https://doi.org/10.18653/v1/2020.sdp-1.11, https://aclanthology.org/2020.sdp-1.11
25. Nair, V., Hinton, G.E.: Rectified linear units improve restricted boltzmann machines. In: ICML (2010)
26. Nallapati, R.M., Ahmed, A., Xing, E.P., Cohen, W.W.: Joint latent topic models for text and citations. In: Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining. pp. 542–550 (2008)
27. Pagliardini, M., Gupta, P., Jaggi, M.: Unsupervised learning of sentence embeddings using compositional n-gram features. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). pp. 528–540. Association for Computational Linguistics, New Orleans, Louisiana (Jun 2018). https://doi.org/10.18653/v1/N18-1049, https://www.aclweb.org/anthology/N18-1049
28. Pennington, J., Socher, R., Manning, C.: GloVe: Global vectors for word representation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 1532–1543. Association for Computational Linguistics, Doha, Qatar (Oct 2014). https://doi.org/10.3115/v1/D14-1162, https://aclanthology.org/D14-1162
29. Ramos, J., et al.: Using tf-idf to determine word relevance in document queries. In: Proceedings of the first instructional conference on machine learning. vol. 242, pp. 133–142. New Jersey, USA (2003)
30. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Now Publishers Inc (2009)
31. Saier, T., Färber, M.: unarXive: a large scholarly data set with publications' full-text, annotated in-text citations, and links to metadata. Scientometrics **125**(3), 3085–3108 (Dec 2020). https://doi.org/10.1007/s11192-020-03382-z, https://doi.org/10.1007/s11192-020-03382-z
32. Schroff, F., Kalenichenko, D., Philbin, J.: Facenet: A unified embedding for face recognition and clustering. In: 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 815–823 (2015). https://doi.org/10.1109/CVPR.2015.7298682
33. Strohman, T., Croft, W.B., Jensen, D.: Recommending citations for academic papers. In: Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval. pp. 705–706 (2007)
34. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in neural information processing systems. pp. 5998–6008 (2017)

35. Voorhees, E.M.: The trec-8 question answering track report. In: In Proceedings of TREC-8. pp. 77–82 (1999)